Más allá del GIL: Aceleración de algoritmos genómicos con Numba, Cython y CUDA

PyCon España 2026 – Barcelona

Pablo Oca Galeano

8 de Noviembre de 2026

Sobre mí



- **Ingeniería Informática**
  Universidad Pablo de Olavide (Sevilla).
- **Erasmus en HPC**
  LUT University (Finlandia).
- **Graduate Research Assistant**
  Grupo de investigación SynergIA.
- **Software Engineer**
  NTT Data.

Guión general

¿Cuál es el límite de Python cuando nos enfrentamos a problemas de alta intensidad computacional en bioinformática?

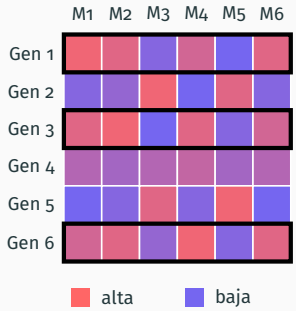
1. Problemas y algoritmos
2. Implementación real
3. Estrategia experimental
4. Resultados estadísticos
5. Conclusiones e ingeniería

BLOQUE 1/5

El reto de la intensidad computacional

Contexto biológico, formulación matemática y el cuello de botella del GIL.

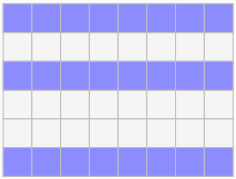
Contexto biológico: la matriz de expresión génica



- **Fila:** un gen.
- **Columna:** una muestra (paciente, tejido, tratamiento…).
- **Celda:** cuánto se *expresa* el gen en esa muestra.
- **Origen:** microarrays y RNA-seq.
- **Escala:** $\sim$20 000 genes $\times$ cientos o miles de muestras.

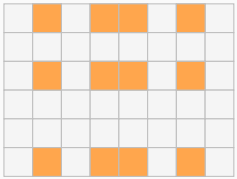
Clustering vs. Biclustering

Clustering



Genes parecidos en **todas** las muestras

Biclustering



Genes parecidos **solo en algunas** muestras

- **Para qué sirve:** módulos de genes co-regulados, subtipos de enfermedad, biomarcadores, respuesta a fármacos.

BiBit y BCCA: ¿qué buscan?

BiBit (Rodríguez-Baena et al., 2011)

- Datos **binarios**: gen activo / inactivo.
- Busca genes que se **encienden a la vez** en las mismas muestras.

BCCA (Bhattacharya y De, 2009)

- Datos **continuos**: nivel de expresión.
- Busca genes que **suben y bajan juntos** (correlación alta).

Desafíos del biclustering

Del problema

- Complejidad de los datos
- Métodos
- Validación

De los datos

- Volumen
- Variedad
- Veracidad

Limitaciones del Big Data (Hadoop/Spark)

- **Arranque en frío**
- **Incompatibilidad con MapReduce**
- **Sincronización de red**
- **Cuello de botella en disco**

Los algoritmos: BiBit y BCCA

BiBit (Binarios):

- Lógica *bitwise*
- Patrones frecuentes
- Alta eficiencia

BCCA (Continuos):

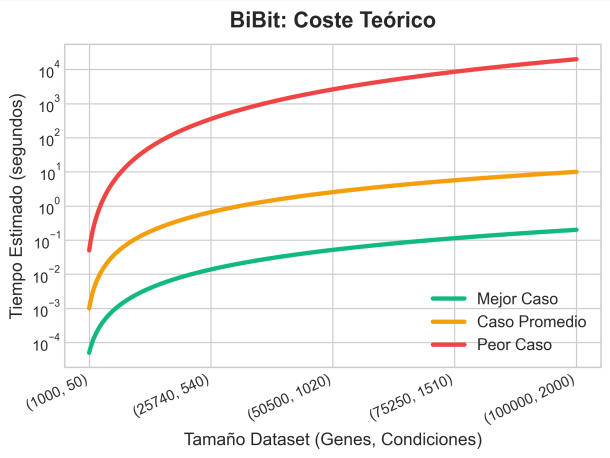
- Correlación numérica
- Alta carga aritmética
- Refinamiento destructivo

Cuello de Botella
Sobrecarga del intérprete y contención del GIL.

Complejidad algorítmica

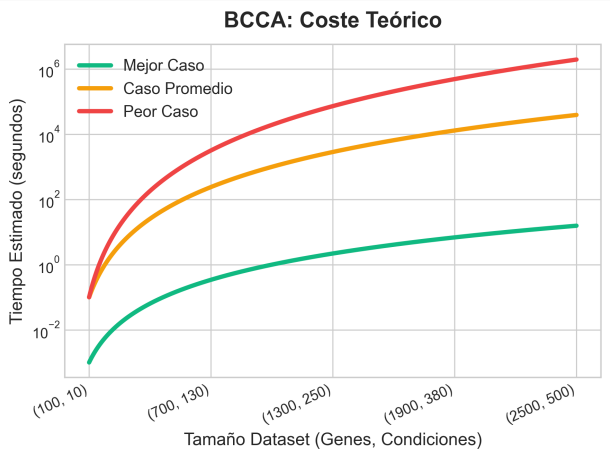
Escenario	BiBit	BCCA	Algoritmos Biclustering
Mejor Caso	$\mathcal{O}(N \cdot M)$	$\mathcal{O}(N^3)$	$\mathcal{O}(N \cdot M)$
Caso Promedio	$\mathcal{O}(N^2)$	$\mathcal{O}(N^4)$	Heurísticas polinómicas
Peor Caso	$\mathcal{O}(N^2 \cdot M)$	$\mathcal{O}(N^4)$	**NP-Hard ($\mathcal{O}(2^N)$)**

Proyección del coste: BiBit



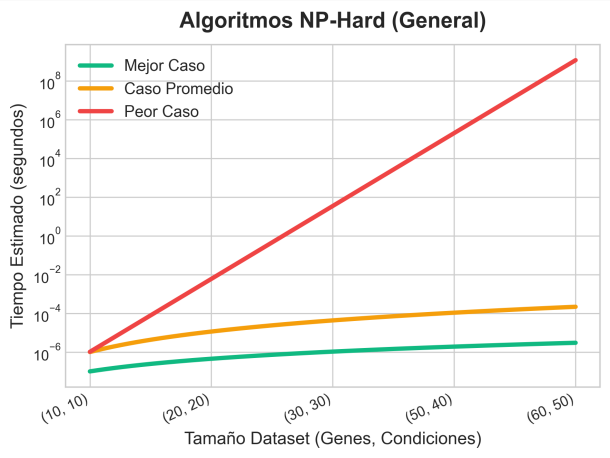
- **BiBit ($\mathcal{O}(N^2)$):** Crecimiento polinómico asimilable.

Proyección del coste: BCCA



- **BCCA ($\mathcal{O}(N^4)$):** Saturación rápida en CPUs.

Proyección del coste: NP-Hard



- **NP-Hard ($\mathcal{O}(2^N)$):** Explosión computacional inmediata.

Algoritmo BiBit: extracción y formación

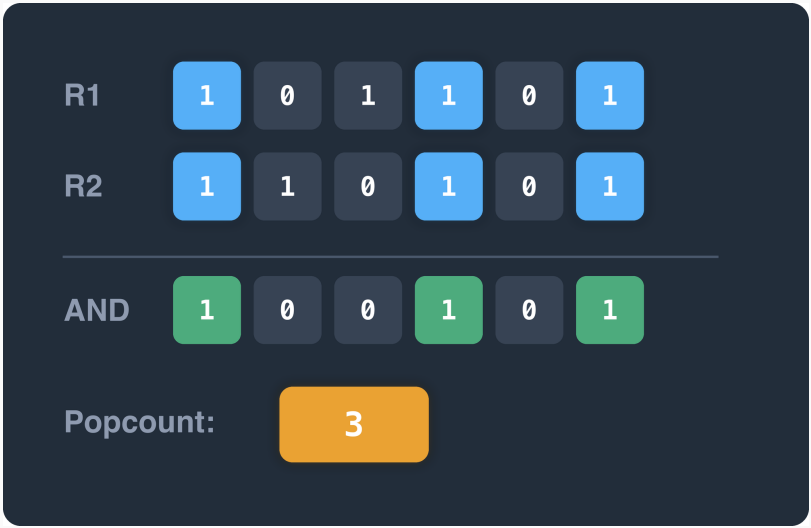
Fase 1: Patrones Frecuentes

- Operaciones *bitwise AND* entre filas.
- Conteo rápido (*popcount*).

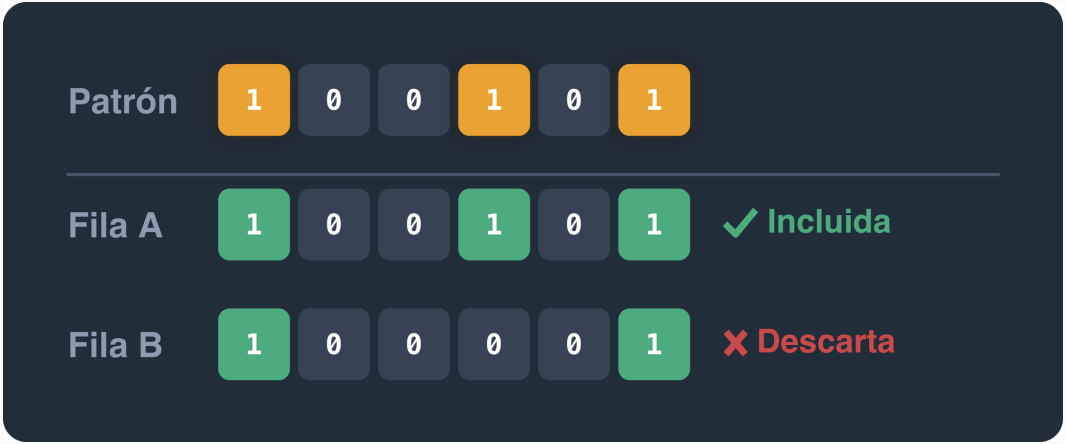
Fase 2: Inclusión de Biclústers

- Validación estricta.
- Consolidación de filas.

Algoritmo BiBit: fase 1, extracción de patrones frecuentes



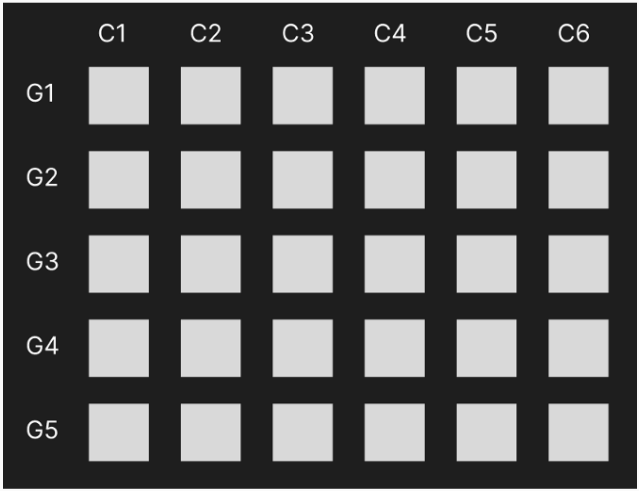
Algoritmo BiBit: fase 2, formación e inclusión de biclústeres



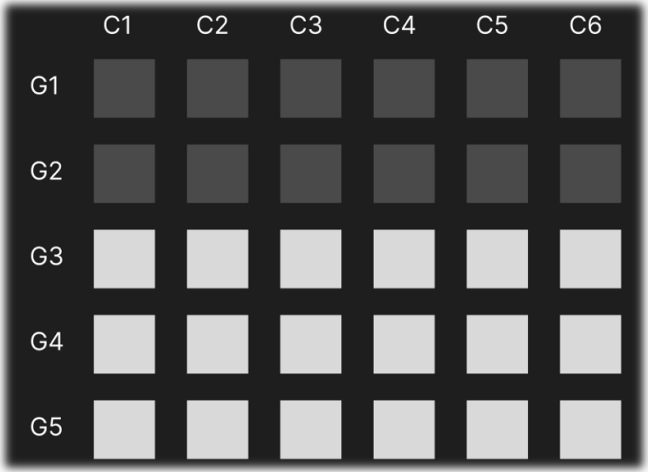
Algoritmo BCCA: filtrado por correlación

- **Enfoque destructivo:** Matriz inicial completa.
- **Poda progresiva:** eliminación de columnas.
- **Convergencia:** biclúster candidato consolidado.

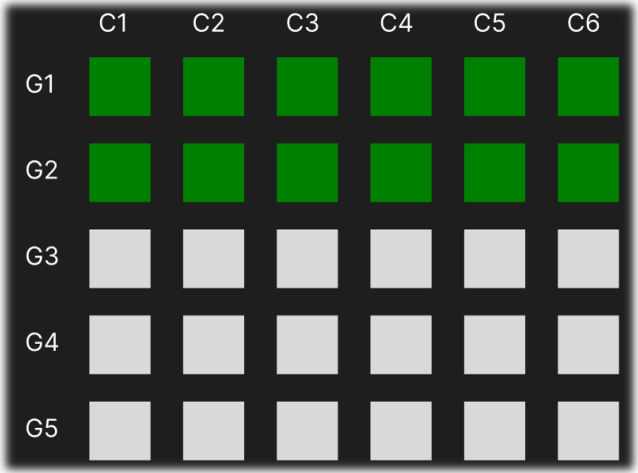
Algoritmo BCCA



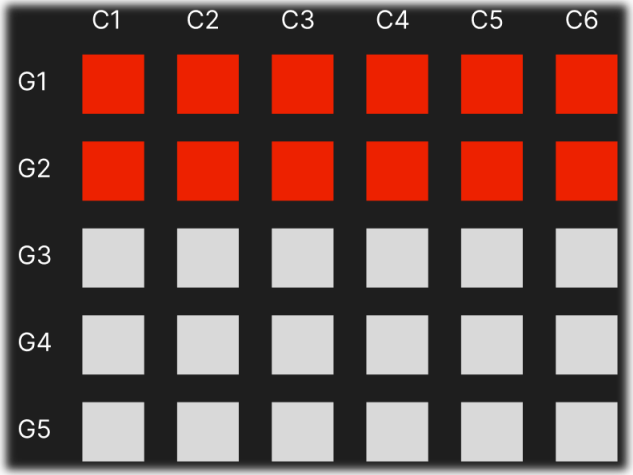
Algoritmo BCCA



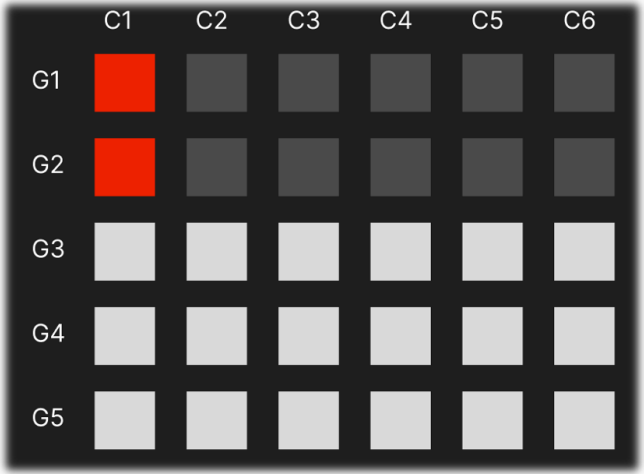
Algoritmo BCCA



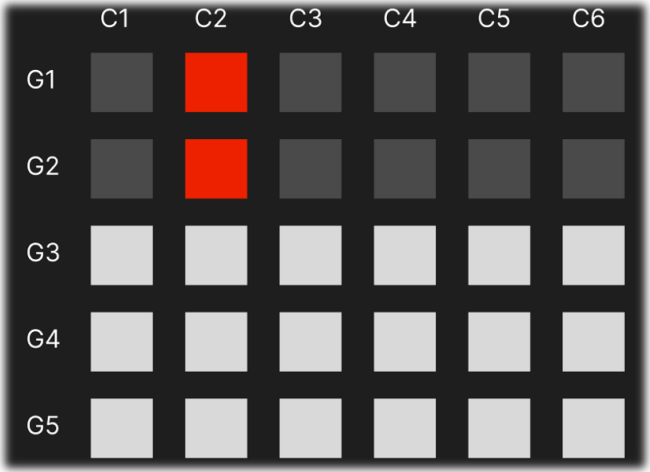
Algoritmo BCCA



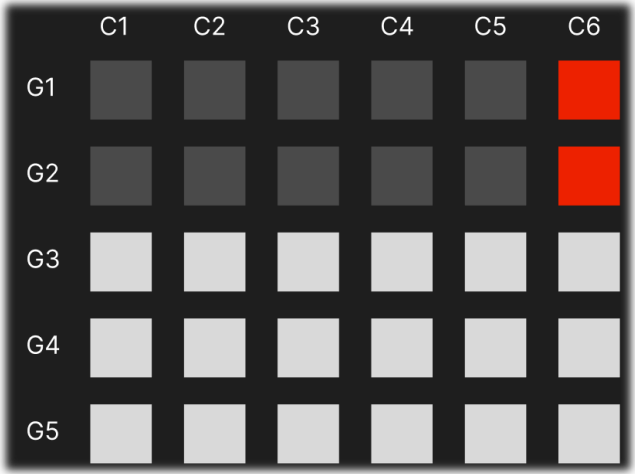
Algoritmo BCCA



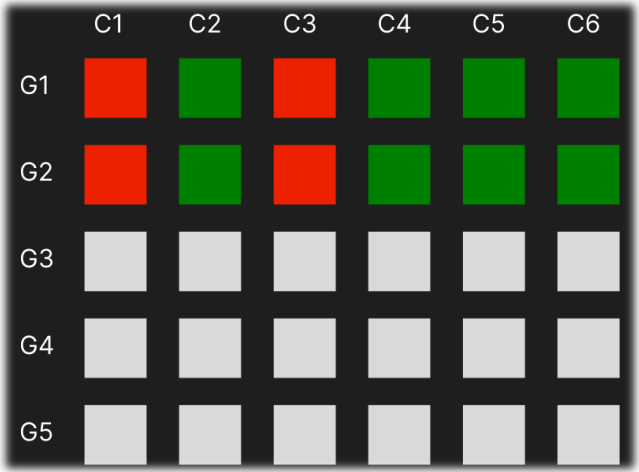
Algoritmo BCCA



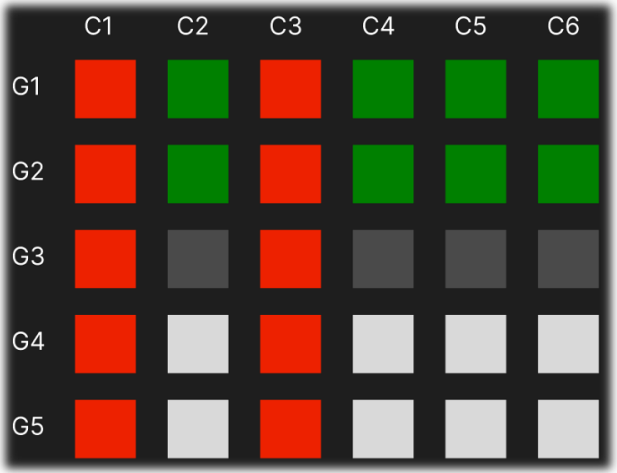
Algoritmo BCCA



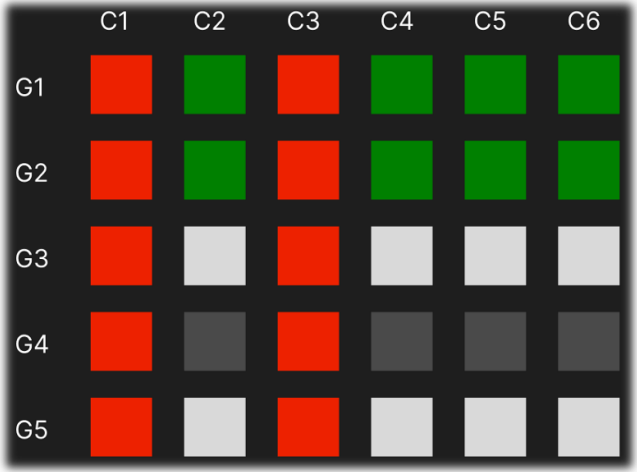
Algoritmo BCCA



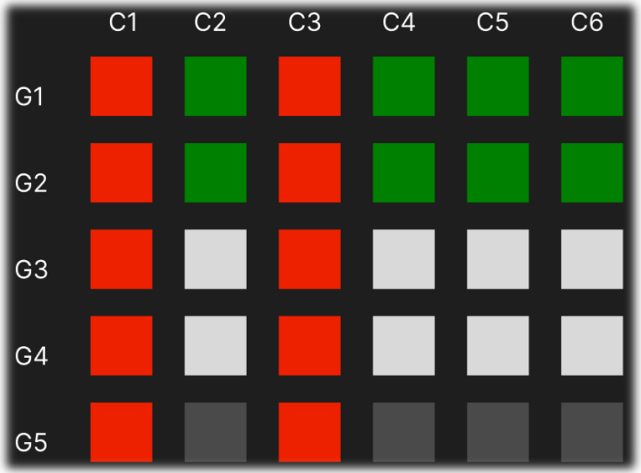
Algoritmo BCCA



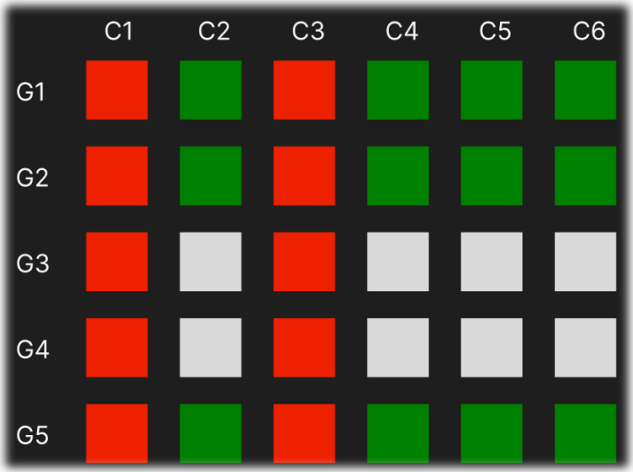
Algoritmo BCCA



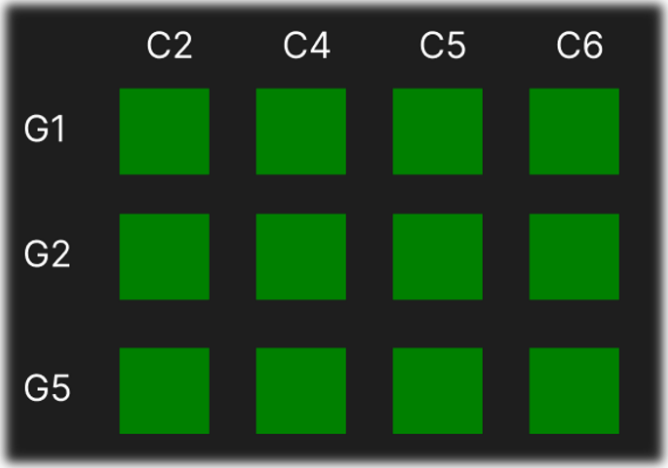
Algoritmo BCCA



Algoritmo BCCA



Algoritmo BCCA



Metodología del benchmark

- **Aceleración estricta:** speedup frente al secuencial.
- **Trade-off:** complejidad y mantenimiento.

Motor multiparadigma: 6 variantes implementadas

CPU:

- Python Secuencial
- Python Numba Paralelo
- Cython Secuencial
- Cython OpenMP

GPU:

- Python Numba CUDA
- Cython CUDA nativo

BLOQUE 2/5

Implementación real

Arquitectura multiparadigma, Numba, Cython y optimización en GPU.

Implementación teórica: BiBit paralelo

CPU

Contar y luego rellenar

Reparto equilibrado

Sin *locks*

GPU

Mapeo hilo $\rightarrow$ pareja

Ecuación cuadrática

`atomicAdd`

Implementación teórica: BCCA paralelo

CPU

Memoria propia por hilo

Correlación en $O(1)$

por columna

GPU

Máscaras de bits

Extracción independiente

BiBit: flujo esencial

```
def processBiBit(dataset, cMnr, cMnc,
                 deviceCount, mode, debug):
    # 64 columnas binarias -> 1 entero uint64
    dataset.data = matrixReduce(dataset.data)
    data = dataset.data
    if mode == 2:
        engine = _NumbaCpuEngine()
    elif mode == 3:
        engine = _NumbaGpuEngine(deviceCount)
    ...           # 4, 5, 6: Cython / OpenMP / CUDA
    else:         # Python + NumPy
        engine = _SequentialEngine()
    # Fase 1: patrones de cada pareja de filas
    patterns = uniquePatterns(engine.patterns(data, cMnc))
    # Fase 2: filas que contienen cada patron
    rowPtr, rowIdx = engine.rows(data, patterns, cMnr)
    return buildModel(dataset, nBits, patterns, ...)
```

BiBit en Numba CPU: núcleo de patrones

```
@njit(inline="always")
def _pairOnes(data, r1, r2, nWords):
    ones = 0
    for w in range(nWords):     # 64 columnas por vuelta
        ones += _popcount64(data[r1, w] & data[r2, w])
    return ones

@njit(parallel=True)
def _nbCountPairs(data, cMnc):
    nRows = data.shape[0]
    perRow = np.zeros(nRows, dtype=np.int64)
    # hilo k: filas k y nRows-1-k (reparto equilibrado)
    for k in prange((nRows + 1) // 2):
        perRow[k] = _countRow(data, k, cMnc)
        other = nRows - 1 - k
        if other != k:
            perRow[other] = _countRow(data, other, cMnc)
    return perRow
```

BiBit en Cython: OpenMP sin GIL

```
cdef extern from *:
    int __builtin_popcountll(unsigned long long x) nogil

cdef inline int _pair_ones(const uint64_t[:, ::1] data,
        Py_ssize_t r1, Py_ssize_t r2,
        Py_ssize_t n_words) noexcept nogil:
    cdef int ones = 0
    for w in range(n_words):
        ones += __builtin_popcountll(data[r1, w] & data[r2, w])
    return ones

# 1) cada fila cuenta sus parejas validas (OpenMP)
for r1 in prange(n_rows, nogil=True,
                 schedule='dynamic', chunksize=4):
    per_row[r1] = _count_row(data, r1, mnc)
# 2) suma acumulada -> tramo de salida de cada fila
# 3) segunda pasada: cada hilo rellena su tramo
```

BiBit en CUDA: de hilo a pareja de filas

```
__global__ void kernel_patterns(const uint64_t *data,
    int n_rows, int n_words, int mnc, long long offset,
    long long size, uint64_t *out,
    unsigned long long *counter) {
  long long t = (long long)blockIdx.x * blockDim.x
              + threadIdx.x;
  if (t >= size) return;
  long long r1, r2;  // r = n-2-floor((sqrt(8(T-1-idx)+1)-1)/2)
  pair_from_index(offset + t, n_rows, &r1, &r2);
  const uint64_t *a = data + r1 * n_words;
  const uint64_t *b = data + r2 * n_words;
  int ones = 0;
  for (int w = 0; w < n_words; w++)
    ones += __popcll(a[w] & b[w]);
  if (ones >= mnc) {  // compactacion: solo los validos
    unsigned long long pos = atomicAdd(counter, 1ULL);
    uint64_t *dst = out + pos * n_words;
    for (int w = 0; w < n_words; w++) dst[w] = a[w] & b[w];
  }
}
```

BCCA en Python: flujo y refinamiento

```
def _bccaSequential(data, threshold, minCols):
    for i in range(numRows):
        for j in range(i + 1, numRows):
            active = np.arange(numCols)
            x, y = data[i], data[j]
            while True:
                m = active.size
                a, b = x - x.mean(), y - y.mean()
                sxx, syy, sxy = a @ a, b @ b, a @ b
                corr = abs(sxy) / math.sqrt(sxx * syy)
                if corr >= threshold or m <= minCols:
                    break
                # |r| sin cada columna, todas a la vez: O(m)
                f = m / (m - 1)
                cxx, cyy = sxx - f*a*a, syy - f*b*b
                cand = np.abs(sxy - f*a*b) / np.sqrt(cxx*cyy)
                keep = np.ones(m, dtype=bool)
                keep[np.argmax(cand)] = False
                active, x, y = active[keep], x[keep], y[keep]
```

BCCA en Cython: tipado estático y memoria en C

```
# cython: boundscheck=False, wraparound=False
from libc.stdlib cimport malloc, free

cdef int _refine_pair(const double* xi, const double* yj,
        int n_cols, double threshold, int min_cols,
        double* xs, double* ys, int* idx) noexcept nogil:
    ...  # el mismo algoritmo, con bucles de C

# run_bcca(..., parallel_mode=True): OpenMP
with nogil, parallel():
    # memoria de trabajo propia de cada hilo
    xs = <double*>malloc(n_cols * sizeof(double))
    ys = <double*>malloc(n_cols * sizeof(double))
    idx = <int*>malloc(n_cols * sizeof(int))
    for i in prange(n_rows, schedule='dynamic'):
        _process_row(data, i, threshold, min_cols,
                     valid, masks, xs, ys, idx)
    free(xs); free(ys); free(idx)
```

BCCA: puente Cython y GPU

```
def processBcca(dataset, correlationThreshold,
                minCols, deviceCount, mode, debug):
    # Memoria contigua y float64 para C/CUDA
    data = np.ascontiguousarray(dataset.data,
                                dtype=np.float64)
    ...
    elif mode == 6:   # Cython + CUDA
        valid, masks = bcca_gpu_cython.run_bcca_gpu(
            data, correlationThreshold, minCols,
            GPU_CHUNK_BYTES)
    ...
    # pareja valida -> Bicluster(filas i, j + columnas)
    oModel = buildModel(data.shape[0], data.shape[1],
                        valid, masks)
```

- Mismo patrón con `bcca_cython.run_bcca(data, ..., parallel)`.

El precio oculto: compilar y distribuir

```
def get_cuda_compute_capability():
    result = subprocess.run(
        ['nvidia-smi', '--query-gpu=compute_cap',
         '--format=csv,noheader'],
        capture_output=True, text=True, timeout=5)
    ...
    # Si no se detecta la GPU: sm_75 por defecto
    return "75"
```

- **Numba:** `pip install` y listo.
- **Cython + CUDA:** `nvcc`, arquitectura `sm_XX`, compilador C y OpenMP en la máquina destino.

BLOQUE 3/5

Estrategia experimental

Rigor metodológico, aislamiento en AWS y validación estadística.

Cómo se midió

- **Máquina:** instancia AWS g5.8xlarge, con AMD EPYC 7R32 (32 vCPU) y 128 GB de RAM.
- **Gráfica:** NVIDIA A10G con 24 GB de memoria (CUDA 13.2).
- **Software:** `bioscience` con Python 3.12, Numba 0.68 y Cython 3.3.
- **Aislamiento:** subprocesos independientes (limpieza de RAM).
- **Análisis separado:** independencia entre algoritmos.

Banco de pruebas

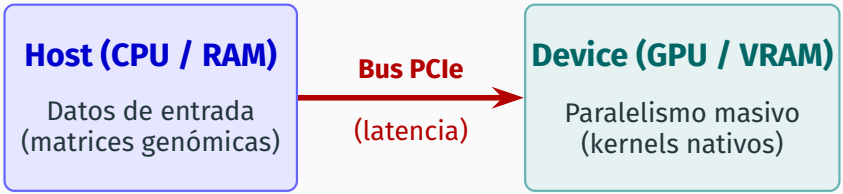
	BiBit	BCCA
Tamaños	1 500×1 500 – 10 000×10 000	50×50 – 1 000×1 000
Geometrías	4 cuadradas + 6 rectangulares	7 cuadradas + 6 rectangulares
Variantes por tamaño	6 densidades (0,3 – 0,8)	6 niveles de ruido
Datasets	60	78
Modos	6	6
Observaciones	360	468

- **736 medidas reales + 92 que superan el límite de 30 min** (se estiman).

Medir bien: trampas evitadas

- **Compilación JIT:** *warm-up* de Numba antes de cronometrar.
- **Fugas entre modos:** un subproceso por ejecución.
- **Ejecuciones eternas:** límite de 30 min por ejecución en todos los modos.
- **Tiempo medido:** solo la llamada al algoritmo, no la lectura del fichero.

Cuello de botella de transferencia: CPU vs. GPU



- **Problemas pequeños:** coste PCIe insalvable.
- **Problemas masivos:** amortización por paralelismo.

Ideas clave del análisis

- **Friedman:** contraste global.
- **Wilcoxon pareado:** consistencia del *baseline*.
- **Kendall W:** estabilidad de ordenamiento.
- **Spearman & Speedup geométrico:** escalabilidad monótona y síntesis de mejora.

BLOQUE 4/5

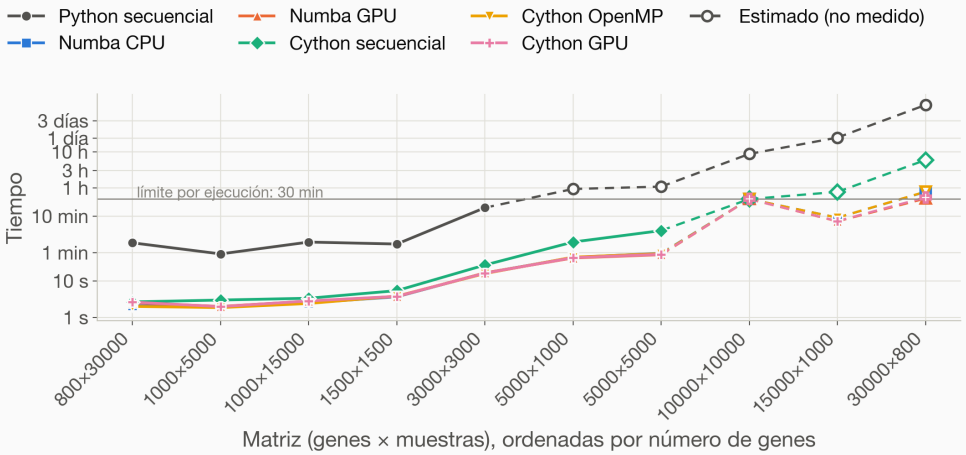
Resultados estadísticos

Análisis de speedups, escalabilidad y dispersión en BiBit y BCCA.

BiBit: lectura sintética

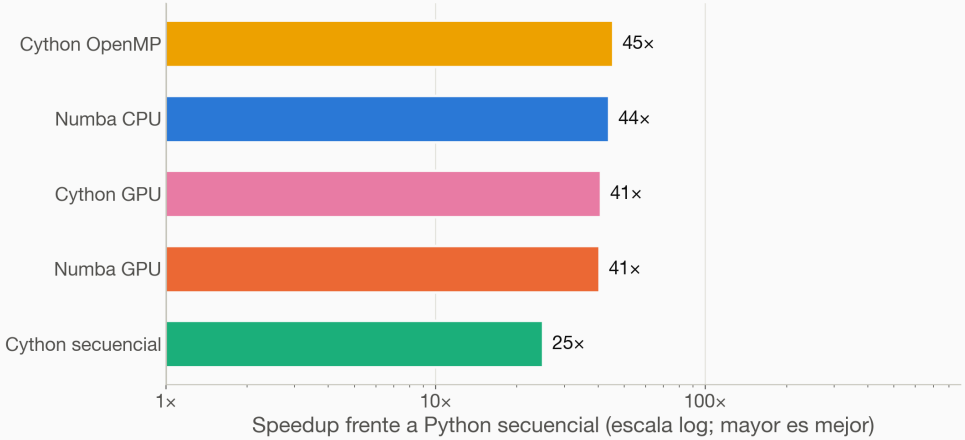
- **Cuatro modos empatados:** Cython OpenMP 45×, Numba CPU 44×, Numba GPU y Cython GPU 41×.
- **La GPU no gana a la CPU:** paralelizar aporta casi todo; la GPU, nada extra.
- **Friedman:** rechazo formal ($p < 0{,}001$).
- **Wilcoxon:** los cinco modos ganan al secuencial en el 100 % de las matrices.

BiBit: tiempo por modo y tamaño



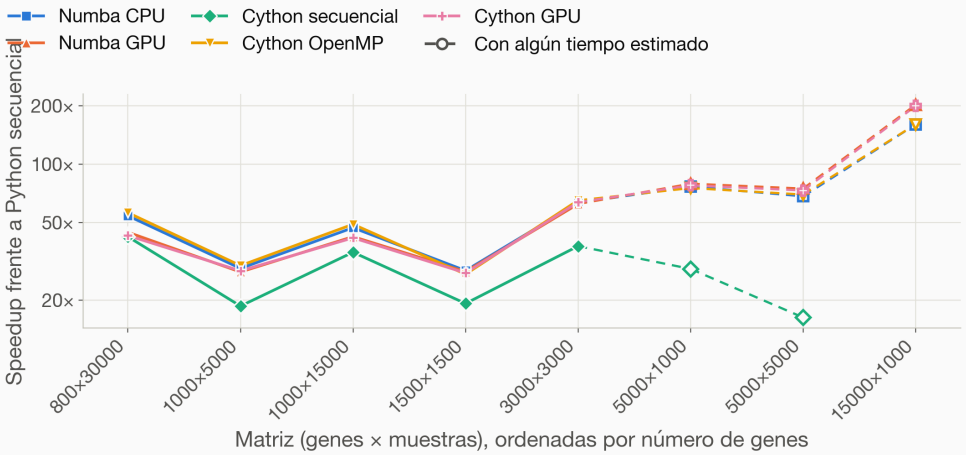
BiBit · densidad 50 % · huecos y discontinua: estimado con un modelo de parejas de genes y biclústeres; si no terminó, ≥ 30 min

BiBit: speedup por modo



Media geométrica sobre 34 matrices con todos los modos medidos · AWS g5.8xlarge (AMD EPYC 7R32, 32 vCPU · NVIDIA A10G)

BiBit: speedup por tamaño

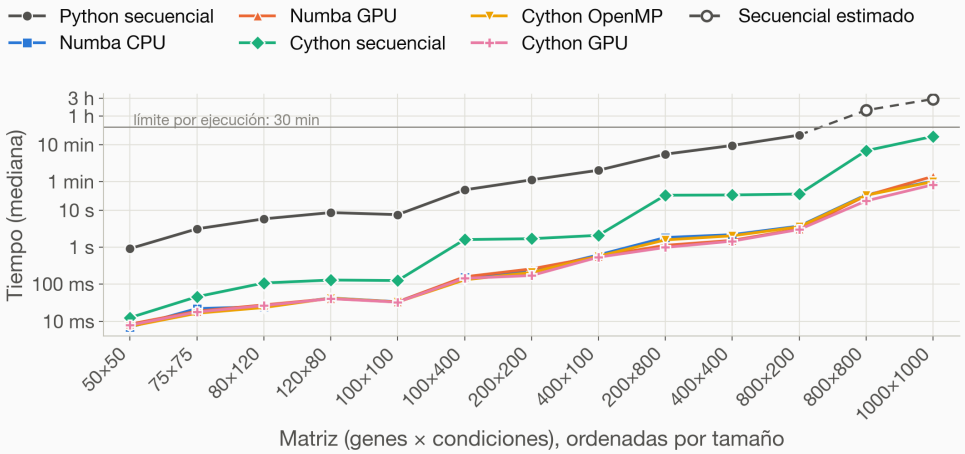


BiBit · densidad 50 % · huecos y discontinua: Python secuencial estimado · sin 10000×10000 ni 30000×800: ningún modo terminó en 3

BCCA: lectura sintética

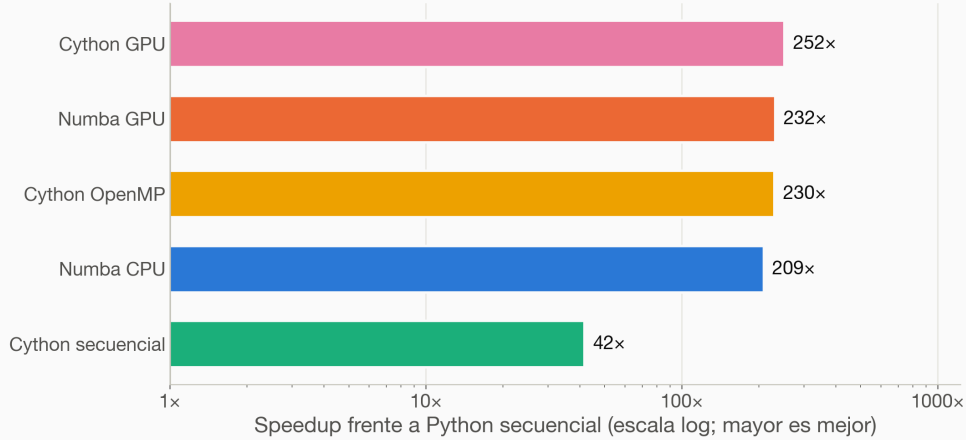
- **Cython GPU:** ganador (252×), seguido de Numba GPU (232×) y Cython OpenMP (230×).
- **Cython secuencial:** 42× sin un solo hilo.
- **Escalado ordenado:** el tiempo crece con el tamaño (Spearman $\rho \approx 0{,}97$).

BCCA: tiempo mediano por modo



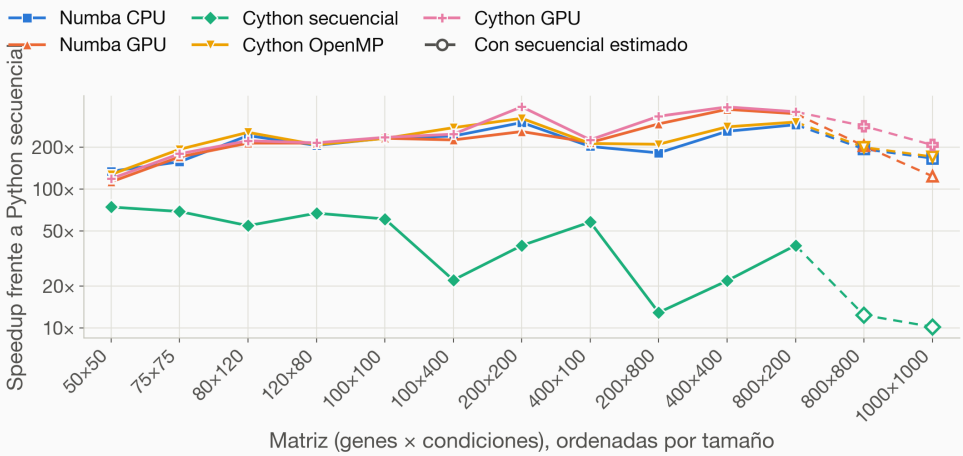
BCCA · mediana de 6 variantes por tamaño · escala logarítmica · AWS g5.8xlarge

BCCA: speedup por modo



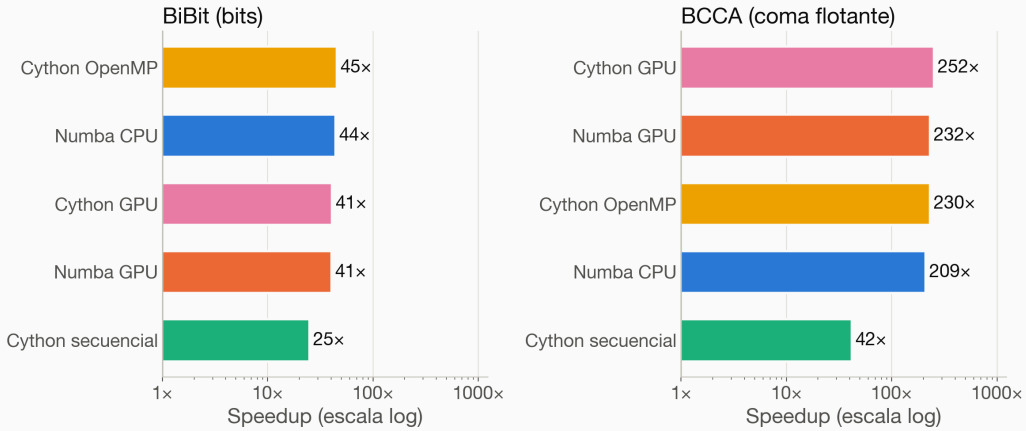
Media geométrica sobre 66 matrices con todos los modos medidos · AWS g5.8xlarge (AMD EPYC 7R32, 32 vCPU · NVIDIA A10G)

BCCA: speedup por tamaño



BCCA · mediana de 6 variantes por tamaño · huecos: el secuencial no terminó en 30 min y se estima

BiBit vs. BCCA: mismo paradigma, distinto efecto



Media geométrica · BiBit: 34 matrices, BCCA: 66 matrices · mismo hardware y mismos seis modos

Complejidad de implementación

Variante	BiBit	BCCA	Herramientas necesarias
Python secuencial	26	40	Python + NumPy
Numba CPU	94	80	+ `numba`
Numba GPU	139	96	+ *driver* CUDA
Cython secuencial	134*	115*	+ compilador C
Cython OpenMP	134*	115*	+ OpenMP
Cython GPU	257	146	+ `nvcc` y CUDA C/C++

Líneas de código propias de cada variante (sin comentarios ni líneas en blanco). *Secuencial y OpenMP comparten el mismo módulo.

- **Numba CPU:** 2–3,6× más código que el secuencial; 44× en BiBit y 209× en BCCA.
- **Cython GPU en BCCA:** ∼3,7× más código y tres lenguajes por 252×.

Resumen estadístico

	BiBit	BCCA
Friedman (χ^2, p)	135,82; $1{,}4 \cdot 10^{-27}$	258,64; $7{,}7 \cdot 10^{-54}$
Kendall W	0,80 (acuerdo fuerte)	0,78 (acuerdo fuerte)
Wilcoxon vs secuencial	$p \approx 1{,}2 \cdot 10^{-10}$ en todos	$p \approx 1{,}6 \cdot 10^{-12}$ en todos
Datasets donde gana el modo	100 %	100 %
Spearman ρ (tamaño–tiempo)	0,17 – 0,38	0,96 – 0,98
Mejor modo	Cython OpenMP (45×)	Cython GPU (252×)

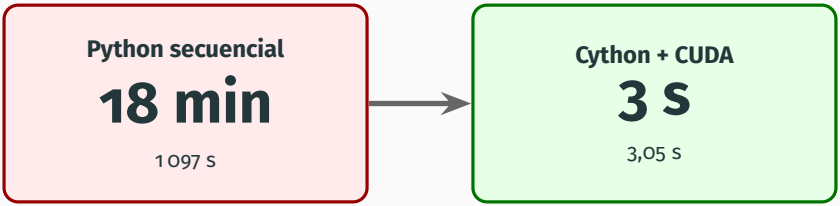
Matrices con los seis modos medidos: 34 en BiBit y 66 en BCCA (límite de 30 min por ejecución).

BLOQUE 5/5

Conclusiones e ingeniería

La respuesta: dónde está el límite de Python y cuándo bajar a C.

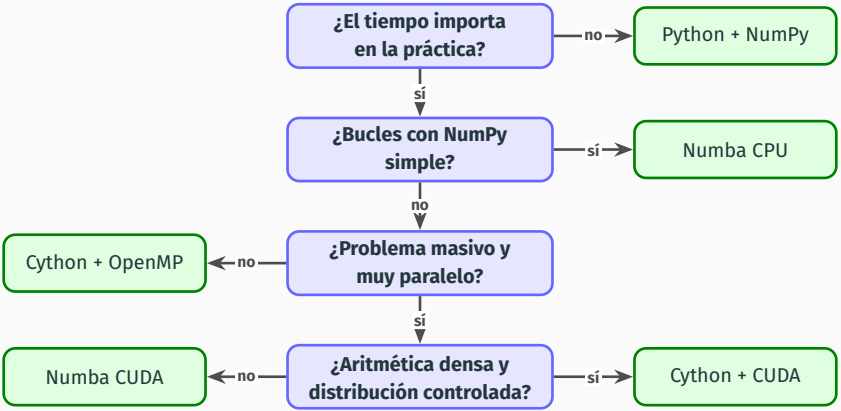
¿Cuál es el límite de Python?



BCCA, matriz real de solo 800 × 200

- **El límite no es el lenguaje:** es el intérprete ejecutando bucles numéricos.
- **Python orquesta, C/CUDA calcula:** el techo pasa a ser el hardware.

Guía rápida de decisión



Bajar a C da miedo.
A veces es imprescindible.

- No hay que reescribirlo todo: solo el núcleo caliente.
- El usuario sigue en Python: `bs.bcca(dataset, mode=6)`.
- BCCA: 146 líneas de Cython + CUDA $\rightarrow$ 252$\times$.
- De un experimento inviable a uno rutinario.

¿Preguntas?

Pablo Oca
pabloocagalo03@gmail.com